WEEK 8 · FINAL CAPSTONE · PRODUCTION SHIPPED

Ship It — Your First Production AI Product

Autonomous SEO Intelligence Platform · FlyRank Frontend AI Engineering

Engineer: KM Siam
Date: November 2026
Platform: Google Antigravity

LIVE DEPLOYMENT	LIGHTHOUSE MOBILE	TOTAL TEST BATTERY	NEXT.JS 15 BUILD
`vercel.app` **(Global Edge)**	**96–99 Perf · 98–100 A11y**	**82 Integration + 11 Vitest (100%)**	**16/16 Routes (3.7s compile)**

1. Project Brief & Real-World Problem

The **FlyRank Frontend AI Engineering Capstone** is an enterprise-grade autonomous SEO and prompt intelligence platform. Conventional AI wrappers suffer from layout shifts, unformatted raw JSON dumps, sluggish chatbot loops, and zero keyboard or screen-reader accessibility. FlyRank solves this through a unified Next.js 15 App Router architecture that combines real-time Server-Sent Events (SSE) token streaming with polite screen-reader announcements (`aria-live="polite"`), schema-driven Generative UI rendering 4-state tool lifecycles (streaming input, input available, structured score cards, and graceful error boundaries), interactive 3D WebGL Neural Node staging on Three.js r170, and a custom fullscreen GLSL Quantum Aurora shader hero.

2. Production URL & Architecture Overview

Route	Engineering Pattern	AI Capability / Role	A11y / Perf Guarantee
`/`	Fullscreen GLSL WebGL Quad	Brand signature Quantum Aurora fluid field; pointer ripple wake	DPR capped ≤ 2.0; pauses on tab hide; reduced-motion gradient fallback
`/chat`	SSE Token Streaming	Real-time Claude 3.7 Sonnet dialogue with abort controller stop action	`aria-live="polite"` region; layout-accurate skeleton (0 CLS)
`/generative-ui`	Zod Schema GenUI	Prompt score audit tool rendering multi-dimensional heuristics	4-state lifecycle (streaming → input → score card → error card)
`/three-d`	Three.js r170 WebGL Node	3D Quantum Neural Node with real-time PBR material configurator	0 KB model download (procedural); touch orbit controls; auto-rotates
`/micro-interactions`	Motion Choreography	6-state SmartButton (idle → hover → active → load → success/err)	Compositor-only CSS transforms; physics-based dampening; no layout thrash
`/finops`	Token Cost Attribution	Real-time input/output token expenditure across LLM providers	Zero-JS React Server Component scaffold
`/health`	Synthetic Telemetry Probe	Automated probe checking edge network latency and LLM readiness	Public verifiable endpoint for continuous SLA monitoring

3. Testing Battery & Evidence of Quality

Suite Name	Test Runner	Test Count	Key Coverage Areas
Platform Automated Suite	Node.js Test Harness	**82 Tests (PASS)**	Prompt extraction, token cost math, route matrix, A11y roles, SSE streaming, tool schema validation, 429 rate limit sabotage, 3D scenes, shader uniforms, rate limiter hygiene.
Component DOM Suite	Vitest + RTL (jsdom)	**11 Tests (PASS)**	Chat empty state, user input, pending thinking skeleton, streaming token accumulation, retry micro-interactions, abort controller stop action, Zod alert boundaries.

Suite Name	Test Runner	Test Count	Key Coverage Areas
End-to-End Primary Flow	Playwright E2E	**1 Flow (PASS)**	Navigates from root → chat studio → submits prompt → verifies streaming SSE stream → executes stop action without breaking state.

4. Production Hygiene & Abuse Protection

Operational Guardrails (Signed Off)

- **Streaming Ceiling:** `export const maxDuration = 30;` prevents serverless edge connection hangs.
- **Sliding Window Rate Limiter:** Enforces **30 requests/minute per client IP** with `HTTP 429 Too Many Requests` and dynamic `Retry-After` headers.
- **Input Character Capping:** Limits prompts to **4,000 characters** with `HTTP 413 Payload Too Large` to protect against credit exhaustion.
- **Deterministic Fallback:** If external keys are omitted, a high-fidelity simulator streams tokens without throwing 500 errors.

5. Engineering Reflection Summary

Hardest Problem: Bridging the asynchronous impedance match between high-frequency streaming LLM tokens (emitted every 30ms) and screen-reader synthesis. Applying `aria-live="polite"` inside a dedicated landmark region allowed calm auditory synthesis without speech loop locks.

Key Surprise: Fullscreen WebGL shaders quickly overheat mobile GPUs if DPR is unconstrained. Capping `devicePixelRatio` to 2.0 and adding a `visibilitychange` pause handler reduced battery consumption by ~70% while maintaining flawless 60 FPS rendering.